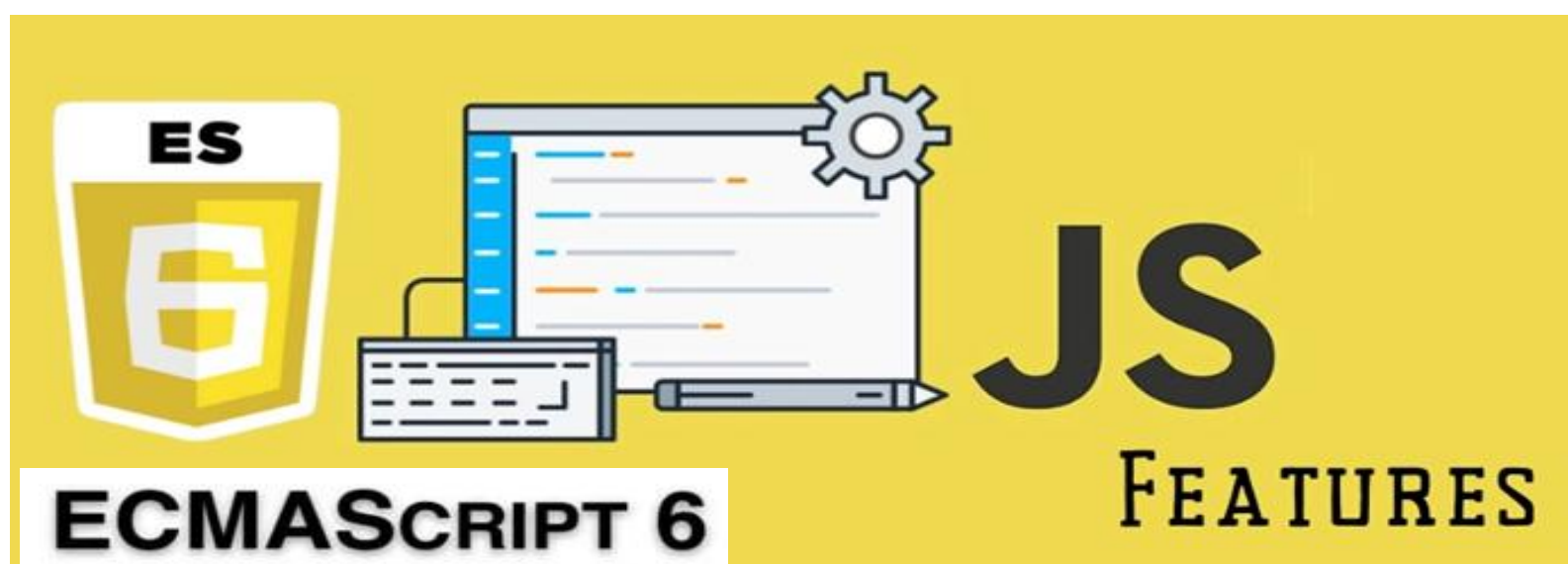




VIT[®]
AP



WEB TECHNOLOGIES

CSE-4004

MODULE 4: ES6

By: Dr.MD.Sirajuddin

SCOPE

WHAT IS ES6 ?

- ES6 or ECMAScript 2015, is the 6th edition of the JavaScript programming language standard, released in June 2015.
- It represents a massive update, introducing modern syntax and features like arrow functions, classes, let/const, modules, and promises.
- These features make code more concise, readable, and efficient for developing complex applications.
- ES5 or ECMAScript 5 is the 5th edition of the ECMAScript programming language standard, released in 2009.
- Also called JavaScript 5, it brought major improvements to the language, including the "use strict" directive, new array methods (e.g., map, filter), Object methods and JSON support.

ES5 V/S ES6

ES5

It is released is 2009.

It accepts string, integer, Boolean, null, and undefined primitive data types.

var keyword has been used, only one way to define variables.

Line of code is more which effect performance

Object manipulation time taking and more line of code.

ES6

It is released is 2015.

It introduced the 'symbol' basic data type to support unique values.

Let and const introduced which improved code writing.

Less line of code improved performance

Object manipulation is possible in less time and less line of code.

ES5 V/S ES6

Feature	ES5	ES6
Year Released	2009	2015
Variable Declaration	var	let, const
Scope	Function-scoped	Block-scoped
Re-declaration	Allowed (var)	Not allowed (let, const)
Functions	Normal functions only	Arrow functions (=>)
Classes	Function constructors	class keyword
String Handling	"Hello " + name	`Hello \${name}` String Interpolation
Default Parameters	Not supported	Supported
Destructuring	Not available a=arr[0]; b=arr[1]; c=arr[2]	Available let arr = [10, 20, 30]; let [a, b, c] = arr;

ES5 V/S ES6

Feature	ES5	ES6
Modules	Not built-in	import / export <code>import { add } from './math.js'; console.log(add(2, 3));</code>
Promises	Not built-in	Built-in support
Spread Operator	Not available <code>var x=arr1.concat(arr2);</code>	Available (...) <code>const arr1 = [1, 2, 3]; const arr2 = [...arr1]; const merged = [...arr1,...arr2];</code>
Strict Mode	Manual ("use strict") <code>"use strict"; x = 20; Error: x is not defined</code>	Automatic in modules <code>// ES6 module or class x = 20; // ReferenceError: x is not defined</code>

SCOPE



Scoping - var

- Function-scoped
- Can be re-declared
- Can be updated

```
• var x = 10;  
  var x = 20;
```

// Allowed

```
• function test() {  
  if(true)  
  {  
    var a = 5;  
  }  
  console.log(a); // 5  
} (Accessible outside  
block)
```

Scoping - let

- Block-scoped
- Cannot be re-declared
- Can be updated

```
• let x = 10;  
  x = 20; // Allowed
```

```
• function test()
```

```
• {  
  if(true)  
  {  
    let a = 5;  
  }  
  console.log(a);  
}  
// Error
```

Scoping - const

- Block-scoped
- Cannot be re-declared
- Cannot be updated

```
• const x = 10;  
  x = 20; // Error
```

const means the reference to the array cannot be changed, but the elements inside the array can be modified.

```
const a = [10, 20, 30];  
a[2] = 50;
```

Arrow function

- Concise way to write functions in ES6.
- Uses **=>** operator
- Anonymous function

let variable_name = (parameters) => expression;

let multiply = (a, b) => a * b;

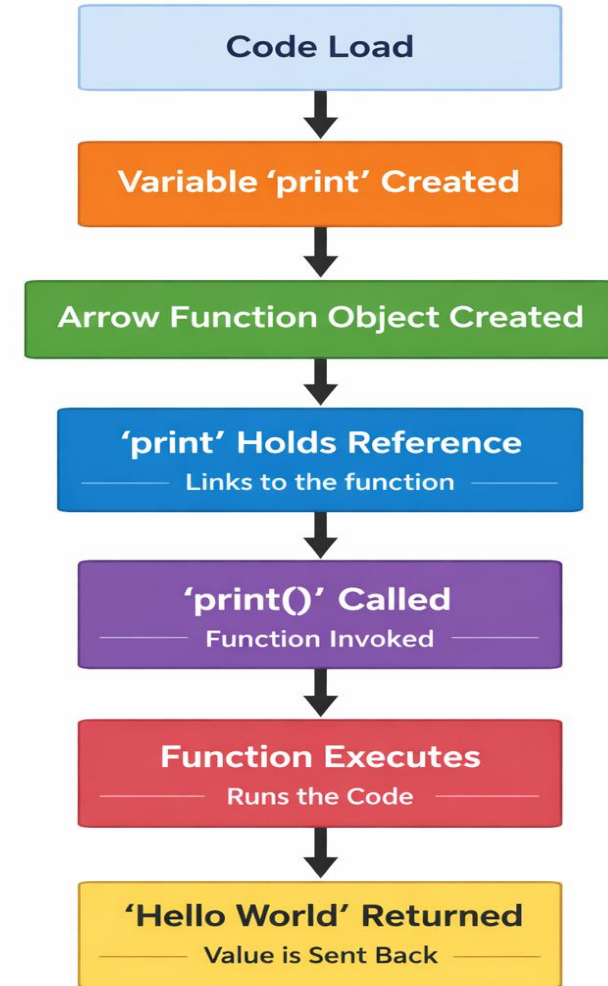
console.log(multiply(5, 3));

Arrow Function

- *Arrow function is a shorthand notation for writing function expression.*
- *It doesn't need **function** & **return** keywords and **Curley braces**.*
- *Arrow function is defined by using =>*
- ***let add = (a,b) => a+b; // with parameters**
alert(add(1,2));*
- ***let print = () => "Hello World"; // without parameters**
alert(print());*
- ***let cube = s => s*s*s; // without parenthesis**
alert(cube(2));*

Arrow Function Execution Process

let print = () => "Hello World";



Default Parameters

- Allows default values for function parameters.

```
function greeting(name = "Student")
```

```
{
```

```
  console.log("Hello " + name);
```

```
}
```

```
greeting();
```

If no value is passed → default is used.

Destructuring

- Extract values from arrays/objects easily.

- Array Destructuring :

```
let arr = [10, 20, 30];  
let [a, b, c] = arr;  
console.log(a); // 10
```

Extraction basis: accessed based on position(index)

- Object Destructuring:

```
let person = {name: "Bob", age: 22};  
let {name, age} = person; //key_name must be specified  
console.log(name);
```

Extraction basis: accessed based on property name(key)

Higher Order Functions

- Functions that take other functions as arguments.
 - Returns a function.
- A function is passed as a parameter to another function.

```
function sayHello(name)
{
  return "Hello " + name;
}
```

```
function display(messageFunction)
{
  console.log(messageFunction("SCOPE"));
}

display(sayHello); //calling
sayHello);
```

SCOPE

Callback Function: A function passed as an argument

```
function user_input(display) //Higher order function  
{  
  const Institute="VIT-AP";  
  display(Institute);  
}
```

```
function showInstitute(name) // callback function  
{  
  alert("Institute Name:"+name);  
}
```

```
user_input(showInstitute); //function call
```

➤ Callbacks are widely used in JavaScript events, timers, and API calls.

Higher Order Functions

➤ A function returns another function.

```
function greetMessage() //returning arrow function
```

```
{  
  return () => {  
    console.log("Good Morning Students");  
  };  
}
```

```
let greet = greetMessage(); //returned function is stored - greet  
greet();
```

Higher Order Functions: Built in

- **map**

```
let numbers = [1,2,3];  
let squares = numbers.map(n => n*n);  
console.log(squares) // [1, 4, 9]
```

- **filter**

```
const num= [1,2,3,4];  
const evens = num.filter(n => n % 2 === 0);  
console.log(evens) // [2, 4]
```

Higher Order Functions: Built in

- **reduce()** reduces an array to single value by applying a function on each element of an array.
- It uses an accumulator.

Syntax: `array.reduce((acc, cur) => acc + cur);`

`acc` points to first element in array

`cur` points to second element in array

Example 1

```
let a=[1,2,3];
```

```
b=a.reduce((acc, cur) => acc + cur); // acc=1, cur=2 (element)
```

```
console.log(b)
```

Output: 6

Example 2

```
b=a.reduce((acc, cur) => acc + cur,0); // acc=0, cur=1 (ele)
```

Output: 6

Defining Class, Constructor & Accessing Data Members

- ES6 introduced class keyword to define a class.

```
class Person
```

```
{  
  constructor(name, age)
```

```
{  
    this.name = name; // this refers to current object  
    this.age = age;  
}
```

```
  greet( )
```

```
{  
    console.log("Hello " + this.name);  
}  
}
```

```
let p1 = new Person("Bob", 22); //accessing data members
```

Accessing Properties

- `console.log(p1.name);`
- `console.log(p1.age);`

Constructor Types

1. Default Constructor created using `constructor()`.
2. Parameterized Constructor

```
constructor(name, age)  
{  
    this.name = name;  
    this.age = age;  
}
```

If no constructor is defined JS uses `constructor() { }`

Inheritance

```
class Person
```

```
{  
  constructor(name)  
  {  
    this.name = name;  
  }  
}
```

```
  introduce()  
  {
```

```
    console.log(this.name + " is a  
    person");  
  }  
}
```

```
class Student extends Person
```

```
{  
  study()  
  {  
    console.log(this.name + " is  
    studying");  
  }  
}
```

```
class Faculty extends Person
```

```
{  
  teach()  
  {  
    console.log(this.name + " is  
    teaching");  
  }  
}
```

Inheritance

// Creating objects

```
let s1 = new Student("Bob");  
let f1 = new Faculty("Alice");
```

// Calling methods

```
s1.introduce(); // Inherited from Person  
s1.study(); // Student method
```

```
f1.introduce(); // Inherited from Person  
f1.teach(); // Faculty method
```

Note: Even though Student does not have its own constructor, JavaScript automatically creates

```
constructor(...args)  
{  
  super(...args);  
}
```

OUTPUT

Bob is a person

Bob is studying

Alice is a person

Alice is teaching

Super

- ✓ Super is used in child class
- ✓ Used to call parent class constructor
- ✓ Used to access parent class method

```
class Person
{
  constructor(name)
  {
    this.name = name;
  }

  introduce()
  {
    console.log(this.name + " is a person");
  }
}
```

```
class Student extends Person
{
  constructor(name)
  {
    super(name); // 👉 Calling Person
    constructor
  }
  study()
  {
    console.log(this.name + " is studying");
  }
}
```

Super: Parent class Constructor

```
class Person
{
  constructor(name)
  {
    this.name = name;
  }

  introduce()
  {
    console.log(this.name + " is a person");
  }
}
```

```
class Student extends Person
{
  constructor(name)
  {
    super(name); // Parent constructor
  }
  study()
  {
    console.log(this.name + " is studying");
  }
}

let s1 = new Student("Siraj");
s1.introduce();
s1.study();
```

Super: Parent class method

```
class Person
{
  constructor(name)
  {
    this.name = name;
  }

  introduce()
  {
    console.log(this.name + " is a
    person");
  }
}
```

class Student extends Person

```
{
  constructor(name)
  {
    super(name); // Parent constructor
  }
  study()
  {
    super.introduce(); //Parent method
    console.log(this.name + " is
    studying");
  }
}
```

```
let s1 = new Student("Bob");
s1.study();
```

Output → Bob is a person
Bob is studying

FAQs

An online shopping application maintains a list of products in a cart. Each product has a **name**, **price**, and **quantity**. The developer needs to calculate the **total price** of items in the cart. To make the code shorter and more readable, the developer decides to use an ES6 arrow function.

Determine high order built in method that can be used to compute **total price** of items.

FAQs

An online shopping application maintains a list of products in a cart. Each product has a **name**, **price**, and **quantity**. The developer needs to calculate the total price of items in the cart. To make the code shorter and more readable, the developer decides to use an ES6 arrow function.

```
let cart = [ {name: "Laptop", price: 50000, quantity: 1},  
  {name: "Mouse", price: 500, quantity: 2},  
  {name: "Keyboard", price: 1500, quantity: 1} ];
```

```
let totalCost = cart.reduce((total, item) => total + (item.price *  
  item.quantity), 0);  
console.log("Total Cost:", totalCost);
```

FAQs

In a Zomato-like food delivery application, customers can apply a coupon code to get a discount on their order. If the customer does not provide any coupon code, the system should apply a default discount of 5%.

Determine the ES6 feature required to implement the given scenario.

FAQs

In a Zomato-like food delivery application, customers can apply a coupon code to get a discount on their order. If the customer does not provide any coupon code, the system should apply a default discount of 5%.

```
function Total_Bill(amount, discount = 0.05)
```

```
{  
  return amount - (amount * discount);  
}
```

```
console.log(calculateBill(1000)); // no coupon
```

```
console.log(calculateBill(1000, 0.20)); // coupon with 20% discount
```

FAQs

In a Zomato food delivery application, the system receives order details from the server in the form of an object containing information such as **customer name**, **restaurant name**, **food item**, **price**, and **delivery address**.

The developer wants to extract **customer name**, **food item** and **price** from the object quickly without repeatedly using the object name. Determine which ES6 feature is required and how to implement it?

FAQs

```
let order = { customerName: "Rahul",  
restaurant: "Pizza Hut",  
foodItem: "Veg Pizza",  
price: 350,  
address: "Vijayawada" };
```

```
let {customerName, foodItem, price} = order;  
console.log("Customer:", customerName);  
console.log("Food Item:", foodItem);  
console.log("Price:", price);
```

FAQs

- A banking system manages different types of accounts such as **Savings Account** and **Current Account**. All accounts share some common properties like account holder **name** and **balance**.
- Instead of writing the same code multiple times, the developer creates a base class **Account**. Other account types like **SavingsAccount** inherit properties and methods from the **Account** class using inheritance and adds the **interest** rate feature.
- The **super** keyword is used to call the parent class constructor so that the common properties can be initialized.

FAQs

```
class Account {  
    constructor(name, balance)  
    {  
        this.name = name;  
        this.balance = balance;  
    }  
  
    display()  
    {  
        console.log("Account Holder:",  
            this.name);  
        console.log("Balance:", this.balance;  
    }  
}
```

```
class SavingsAccount extends Account  
{  
    constructor(name, balance, interestRate)  
    {  
        super(name, balance);  
        this.interestRate = interestRate;  
    }  
    showInterest()  
    {  
        console.log("ROI:", this.interestRate +  
            "%");  
    }  
}
```

FAQs

```
let acc1 = new SavingsAccount("Rahul", 10000, 5);  
acc1.display();  
acc1.showInterest();
```